

从 Test-Time Scaling 到自动科研

Reasoning、Agent、Multi-Agent 与 AutoResearch
雄安 · 深度学习课程 · 第六讲

郭绍阳
清华大学人工智能学院

2026-09-24

课程目录

- ▶ Part 1 · Reasoning 范式下，如何让大模型拥有一个能力
- ▶ Part 2 · LLM (Large Language Model) Reasoning 的规律
- ▶ Part 3 · Agent 的定义和实现
- ▶ Part 4 · Agent Harness 和自我迭代
- ▶ Part 5 · Multi-agent: 能不能堆很多 agent?
- ▶ Part 6 · 并行的极端形态: OpenAI 解决千禧年问题

Reasoning 是在时间维上扩展一个模型，*multi-agent* 是在并行维上扩展一个模型，而 *context engineering* 决定这些额外计算究竟是在协作，还是只是在制造噪声。

Part 1 · Reasoning 范式下，如何让大模型拥有一个能力

— 姚顺雨 《The Second Half》 (2025-04) · <https://ysymyth.github.io/The-Second-Half>

Shunyu Yao

姚顺雨



The Second Half

tldr: We're at AI's halftime.

For decades, AI has largely been about developing new training methods and models. And it worked: from beating world champions at chess and Go, surpassing most humans on the SAT and bar exams, to earning IMO and IOI gold medals. Behind these milestones in the history book — DeepBlue, AlphaGo, GPT-4, and the o-series — are fundamental innovations in AI methods: search, deep RL, scaling, and reasoning. Things just get better over time.

So what's suddenly different now?

In three words: RL finally works. More precisely: RL finally generalizes. After several major detours and a culmination of milestones, we've landed on a working recipe to solve a wide range of RL tasks using language and reasoning. Even a year ago, if you told most AI researchers that a single recipe could tackle software engineering, creative writing, IMO-level math, mouse-and-keyboard manipulation, and long-form question answering — they'd laugh at your hallucinations. Each of these tasks is incredibly difficult and many researchers spend their entire PhDs focused on just one narrow slice.

Yet it happened.

So what comes next? The second half of AI — starting now — will shift focus from solving problems to defining problems. In this new era, evaluation becomes more important than training. Instead of just asking, "Can we train a model to solve X?", we're asking, "What should we be training AI to do, and how do we measure real progress?" To thrive in this second half, we'll need a timely shift in mindset and skill set, ones perhaps closer to a product manager.

The first half

上半场：方法比任务通用

- ▶ 史上最有影响力的论文全是**方法/模型**：AlexNet、Transformer、GPT-3；benchmark 是次要的。
- ▶ 证据：ImageNet 数据集引用不到 AlexNet 的 1/3；WMT'14 引用约 1.3k，而 Transformer > 160k。
- ▶ 旧游戏规则：发明新方法 → 爬 benchmark → 造更难 benchmark → 循环。
- ▶ 为什么方法赢：**方法比任务难、也比任务通用**。

Recipe 如何执行：新 bench 是怎么被刷上去的

1. **新 bench 出现**：评测目标定下，通常可自动验证（数学短答案 / 代码单测）。
2. **prior 是现成的**：基模已蒸馏语言、世界知识与常识——不需要从头造智能。
3. **造相关数据**：按 bench 同分布程序化生成题目，自带可验证奖励（编译器 / 答案校验器）。
4. **做 RL (Reinforcement Learning)**：rollout + 规则奖励，策略梯度（GRPO 族）；模型自己长出长 CoT（Chain-of-Thought）。
5. **bench 被刷上去**：分数上升。全流程工业化——所以「+3% 的小方法」会被下一代基模直接碾过。

—recipe 框架出自姚顺雨 《The Second Half》

下半场：瓶颈从 **optimization** 转到 **evaluation**

- ▶ 若 recipe 已这么强：「我发明小方法，benchmark +3%」越来越没意义——下一代基模一更新直接 +20-30%。
- ▶ 但重点不是「benchmark 不重要」，而是：**benchmark 里隐含的世界假设可能错了**。
- ▶ 假设一「任务应可自动打分、单发自主」vs 现实：人与 agent 全程交互 → Chatbot Arena、tau-bench。
- ▶ 假设二「题目 i.i.d.」vs 现实：程序员对 repo 越做越熟（Google SWE）；有长期记忆的 agent，第 100 个任务与第 1 个**不同分布**。
- ▶ **Utility problem**：棋/Go/SAT (satisfiable) /bar/IOI/IMO 都赢了，GDP 几乎没动——我们到底有没有在测真正有经济/现实价值的能力？

Part 2 · LLM Reasoning 的规律

—Lilian Weng 《Why We Think》(2025-05) · <https://lilianweng.github.io/posts/2025-05-01-thinking/>

Lil'Log

[Posts](#) [Archive](#) [Search](#) [Tags](#) [FAQ](#)

Why We Think

Date: May 1, 2025 | Estimated Reading Time: 40 min | Author: Lilian Weng

► Table of Contents

Special thanks to [John Schulman](#) for a lot of super valuable feedback and direct edits on this post.

Test time compute (Graves et al. 2016, Ling, et al. 2017, Cobbe et al. 2021) and Chain-of-thought (CoT) (Wei et al. 2022, Nye et al. 2021), have led to significant improvements in model performance, while raising many research questions. This post aims to review recent developments in how to effectively use test-time compute (i.e. “thinking time”) and why it helps.

Motivation

Enabling models to think for longer can be motivated in a few different ways.

Analogy to Psychology

The core idea is deeply connected to how humans think. We humans cannot immediately provide the answer for “What's 12345 times 56789?”. Rather, it is natural to spend time pondering and analyzing before getting to the result, especially for complex problems. In [Thinking, Fast and Slow](#) (Kahneman, 2013), Daniel Kahneman characterizes human thinking into two modes, through the lens of the [dual process theory](#):

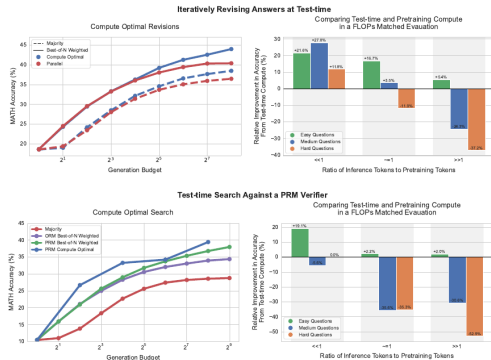
- *Fast thinking (System 1)* operates quickly and automatically, driven by intuition and emotion while requiring little to no effort.

Test-time compute 是一根新的 scaling 轴

- ▶ 旧 scaling law 的轴：参数量 N 、数据 D 、训练 compute C ；现在多一根：推理时愿意给多少 computation。
- ▶ 每 token 计算量 $\approx 2 \times$ 参数量 (MoE 再除 sparsity)；CoT 让模型为每个答案 token 先做大量中间计算。
- ▶ 好性质：计算量随难度自适应——同一模型面对 $1 + 1$ 与 IMO 题不该用一样多的计算。
“CoT has a nice property that it allows the model to use a variable amount of compute depending on the hardness of the problem.”

—Lilian Weng, *Why We Think*

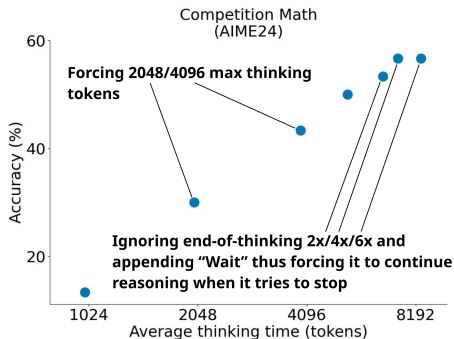
图 · test-time scaling law (Snell et al. 2024)



Snell, Lee, Xu, Kumar. *Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters*, Figure 1 · arXiv:2408.03314

- 核心结论：compute-optimal 的 test-time 分配比 best-of-N 高效 $4\times$ 以上；FLOPs (floating-point operations) 匹配下，小模型 +test-time 在易/中题可胜 $14\times$ 大模型——但难题补不动。

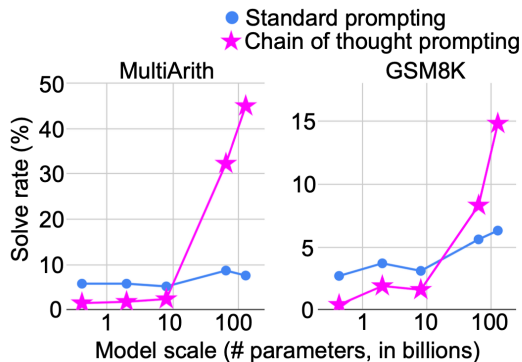
图 · TTS 代表: budget forcing (s1)



Muennighoff et al. *s1: Simple test-time scaling*, budget forcing (追加“wait”拉长思考) · arXiv:2501.19393

- ▶ 方法: 从 Qwen2.5-32B-Instruct 蒸馏约 1000 条长 CoT 微调 7B 得 s1; test-time 用 budget forcing (追加“Wait”续想、“Final Answer:”截断)。
- ▶ 核心结论: thinking tokens 与准确率正相关; 用 rejection sampling 控长度反而越长越差 ⇒ 长度非因果。

图 · TTS (Test-Time Scaling) 代表: CoT 收益随规模放大



Wei et al. 2022 (Chain-of-Thought); 图转引自 Lilian Weng 《Why We Think》blog

- 核心结论: CoT 的收益随模型规模涌现并放大——test-time compute 的回报本身随能力增长。

隐变量视角：采样即搜索

$$p(y | x) = \sum_z p(z | x) p(y | x, z), \quad z = \text{思维链}$$

- ▶ 多条 CoT / 搜索 = 从后验 $p(z | x, y)$ 采样。
- ▶ 由此 best-of-N、beam、self-consistency 被统一成一句话：从后验里采好样本。
- ▶ 也解释了 log-loss 为何仍是自然目标（与预训练一致）。

谱系：Ling'17 AQUA-RAT → Cobbe'21 GSM+verifier → Nye'21 scratchpad → Wei'22 CoT → Kojima'22 “think step by step”；再后来在可自动验证任务（STEM 短答案、代码单测）上做 RL → o1/o3/R1。

arXiv

Search Submit Donate Log in

Computer Science > Computation and Language

[Submitted on 28 Jan 2022 (v1), last revised 10 Jan 2023 (this version, v6)]

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, Denny Zhou

We explore how generating a chain of thought -- a series of intermediate reasoning steps -- significantly improves the ability of large language models to perform complex reasoning. In particular, we show how such reasoning abilities emerge naturally in sufficiently large language models via a simple method called chain of thought prompting, where a few chain of thought demonstrations are provided as exemplars in prompting. Experiments on three large language models show that chain of thought prompting improves performance on a range of arithmetic, commonsense, and symbolic reasoning tasks. The empirical gains can be striking. For instance, prompting a 540B-parameter language model with just eight chain of thought exemplars achieves state of the art accuracy on the GSM8K benchmark of math word problems, surpassing even finetuned GPT-3 with a verifier.

Subjects: **Computation and Language (cs.CL)**; Artificial Intelligence (cs.AI)

Cite as: [arXiv:2201.11903](#) [cs.CL]

(or [arXiv:2201.11903v6](#) [cs.CL] for this version)

<https://doi.org/10.48550/arXiv.2201.11903>

Submission history

From: Jason Wei [view email]

[v1] Fri, 28 Jan 2022 02:33:07 UTC (944 KB)

[v2] Wed, 6 Apr 2022 03:51:50 UTC (933 KB)

[v3] Wed, 1 Jun 2022 00:10:30 UTC (303 KB)

[v4] Mon, 13 Jun 2022 21:44:34 UTC (283 KB)

[v5] Mon, 10 Oct 2022 20:21:17 UTC (285 KB)

[v6] Tue, 10 Jan 2023 23:07:57 UTC (306 KB)

Access Paper:

[View PDF](#)
[HTML \(experimental\)](#)
[TeX Source](#)

 [view license](#)

Current browse context:

cs.CL

[< prev](#) | [next >](#)
[new](#) | [recent](#) | [2022-01](#)

Change to browse by:

[cs](#)
[cs.AI](#)

References & Citations

[NASA ADS](#)
[Google Scholar](#)
[Semantic Scholar](#)

18 blog links

(what is this?)

DBLP - CS Bibliography

[listing](#) | [bibtex](#)

Jason Wei
Xuezhi Wang
Dale Schuurmans
Quoc Le
Denny Zhou

Export BibTeX Citation

Bookmark



Bibliographic Tools

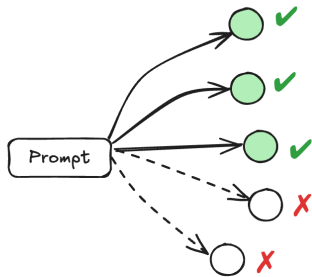
Code, Data, Media Demos Related Papers About arXivLabs

Bibliographic and Citation Tools

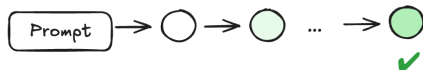
☐ Bibliographic Explorer (What is the Explorer?)

arXiv:2201.11903 · CoT 起点:「越大越想」的实证来源

横向展开: branching 家族



Parallel Sampling



Sequential Revision

- ▶ best-of-N / self-consistency (多数投票) / beam / ToT (Tree of Thoughts) / PRM (Process Reward Model) / REBASE / RATIONALYST; Xie'23 每步自评降累积误差 +5-6%。
- ▶ Wang&Zhou'24: 第一个采样 token 处按 top-1/top-2 置信度差分支、之后 greedy → 自发涌现 CoT。

纵向修改：自纠没有想象可靠（负面结果）

“...this self-correction capability turns out to not exist intrinsically among LLMs ... (1) hallucination, including modifying correct responses to be incorrect; (2) behavior collapse to non-correcting behavior ...or (3) fail to generalize to distribution shift at test time.”

—Lilian Weng, *Why We Think*

- ▶ 「让模型反思一下」不是免费午餐：把对改错、象征性小改、分布越走越偏。
- ▶ 有效的 revision 需要**外部信号**：verifier、环境反馈、unit test、或专门训练的 critic。
- ▶ 训练 corrector：Welleck'23 用 value-improving pairs；SCoRe 两阶段 RL（阶段一 KL (Kullback–Leibler) 只约束第一次作答）防坍缩。
- ▶ Snell'24：易题纯串行更好，难题要串/并最优配比。

arXiv

Search Submit Donate Log In

Computer Science > Computation and Language

[Submitted on 22 Jan 2025 (v1), last revised 4 Jan 2026 (this version, v2)]

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z.F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J.L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiusi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R.J. Chen, R.L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S.S. Li et al. (100 additional authors not shown)

General reasoning represents a long-standing and formidable challenge in artificial intelligence. Recent breakthroughs, exemplified by large language models (LLMs) and chain-of-thought prompting, have achieved considerable success on foundational reasoning tasks. However, this success is heavily contingent upon extensive human-annotated demonstrations, and models' capabilities are still insufficient for more complex problems. Here we show that the reasoning abilities of LLMs can be incentivized through pure reinforcement learning (RL), obviating the need for human-labeled reasoning trajectories. The proposed RL framework facilitates the emergent development of advanced reasoning patterns, such as self-reflection, verification, and dynamic strategy adaptation. Consequently, the trained model achieves superior performance on verifiable tasks such as mathematics, coding competitions, and STEM fields, surpassing its counterparts trained via conventional supervised learning on human demonstrations. Moreover, the emergent reasoning patterns exhibited by these large-scale models can be systematically harnessed to guide and enhance the reasoning capabilities of smaller models.

Subjects: Computation and Language (cs.CL); Artificial Intelligence (cs.AI); Machine Learning (cs.LG)

Cite as: arXiv:2501.12948 [cs.CL]
(or arXiv:2501.12948v2 [cs.CL] for this version)
<https://doi.org/10.48550/arXiv.2501.12948>

Journal reference: Nature volume 645, pages 633–638 (2025)

Related DOI: <https://doi.org/10.1038/s41586-025-09422-z>

Access Paper:

[View PDF](#)
[HTML \(experimental\)](#)
[TeX Source](#)
[view license](#)

Current browse context: **cs.CL**

[< prev](#) | [next >](#)
[new](#) | [recent](#) | [2025-01](#)

Change to browse by:

cs

- [cs.AI](#)
- [cs.LG](#)

References & Citations

[NASA ADS](#)
[Google Scholar](#)
[Semantic Scholar](#)

[Export BibTeX Citation](#)

Bookmark



arXiv:2501.12948 · 四阶段配方与 R1-zero 的一手报告；R1-zero：纯 RL（无 SFT）也涌现长 CoT 与反思

R1 的四阶段配方：逐步拆解

1. **Cold-start SFT (Supervised Fine-Tuning)**：数千条长 CoT 样本（格式规整、语言一致），治语言混杂与可读性，让 RL 从一个「会写思路」的策略起步。
 2. **Reasoning RL**：在可自动验证任务（数学/代码/逻辑）上大规模 RL；奖励 = format (<thinking> 包裹、语言一致) + accuracy (规则校验 / 编译器单测)；GRPO (Group Relative Policy Optimization) 族；涌现反思、回溯、aha moment, thinking tokens 随训练增长。
 3. **Rejection sampling + 非推理 SFT**：从阶段二策略采约 800k 样本（约 600k 推理 + 200k 写作/QA (Question Answering) /角色等），训 2 epoch，把通用能力补回来。
 4. **Final RL**：全任务（推理 + 非推理）再 RL；规则奖励 + reward model (helpful/harmless)，完成对齐收尾。
- 其他方法复现 o1-like 的推理模型：PRM（逐步 rubric 难定、更易 reward hacking）、MCTS (Monte Carlo Tree Search) (token 搜索空间太大) 都未走通。

Faithfulness: CoT 不是天然透明的窗口

- ▶ Lanham'23 三种不忠实：提前下结论；无信息 token 也提分；人类读不懂的编码也提分。
- ▶ Turpin'23：误导性 hint (“A Stanford Professor thinks the answer is X”) 会改答案，但模型不承认——reasoning 模型 (R1、Claude 3.7) 比非 reasoning 更**忠实** (Chen'25)。
- ▶ Baker'25：监测 CoT 能抓 reward hacking (弱模型可监测强模型)；但把 monitor 放进 RL reward → **obfuscated reward hacking**：模型学会**不把坏事写出来**。
- ▶ Yeo'25 长度奖励 → 重复文本 hacking；Chen'25 坏 grader：> 99% 在 hack、< 2% 在 CoT 里承认。

Message: CoT 可监控 $\neq$ CoT 是可靠透明窗口；越直接优化「可读 reasoning」，越可能把窗口优化坏。

论文照片 · Lanham et al. 2023: CoT faithfulness

arXiv

Search Submit Donate Log in

Computer Science > Artificial Intelligence

[Submitted on 17 Jul 2023]

Measuring Faithfulness in Chain-of-Thought Reasoning

Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, Kamilė Lukošiuūtė, Karina Nguyen, Newton Cheng, Nicholas Joseph, Nicholas Schiefer, Oliver Rausch, Robin Larson, Sam McCandlish, Sandipan Kundu, Saurav Kadavath, Shannon Yang, Thomas Henighan, Timothy Maxwell, Timothy Telleen-Lawton, Tristan Hume, Zac Hatfield-Dodds, Jared Kaplan, Jan Brauner, Samuel R. Bowman, Ethan Perez

Large language models (LLMs) perform better when they produce step-by-step, "Chain-of-Thought" (CoT) reasoning before answering a question, but it is unclear if the stated reasoning is a faithful explanation of the model's actual reasoning (i.e., its process for answering the question). We investigate hypotheses for how CoT reasoning may be unfaithful, by examining how the model predictions change when we intervene on the CoT (e.g., by adding mistakes or paraphrasing it). Models show large variation across tasks in how strongly they condition on the CoT when predicting their answer, sometimes relying heavily on the CoT and other times primarily ignoring it. CoT's performance boost does not seem to come from CoT's added test-time compute alone or from information encoded via the particular phrasing of the CoT. As models become larger and more capable, they produce less faithful reasoning on most tasks we study. Overall, our results suggest that CoT can be faithful if the circumstances such as the model size and task are carefully chosen.

Subjects: **Artificial Intelligence** (cs.AI); Computation and Language (cs.CL); Machine Learning (cs.LG)

Cite as: [arXiv:2307.13702](#) [cs.AI]
(or [arXiv:2307.13702v1](#) [cs.AI] for this version)
<https://doi.org/10.48550/arXiv.2307.13702>

Submission history
From: Ethan Perez [[view email](#)]
[v1] Mon, 17 Jul 2023 01:08:39 UTC (492 KB)

Bibliographic Tools Code, Data, Media Demos Related Papers About arXiv Labs

Bibliographic and Citation Tools
☐ Bibliographic Explorer (What is the Explorer?)
☐ Connected Papers (What is Connected Papers?)

Access Paper:
[View PDF](#)
[HTML \(experimental\)](#)
[TeX Source](#)
[view license](#)

Current browse context:
cs.AI
< prev | next >
new | recent | 2023-07
Change to browse by:
cs
cs.CL
cs.LG

References & Citations
[NASA ADS](#)
[Google Scholar](#)
[Semantic Scholar](#)

Export BibTeX Citation

Bookmark
 

arXiv:2307.13702 · 三种不忠实模式的原始实验

arXiv

Search Submit Donate Log In

Computer Science > Artificial Intelligence

[Submitted on 14 Mar 2025]

Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation

Bowen Baker, Joost Huizinga, Leo Gao, Zehao Dou, Melody Y. Guan, Aleksander Madry, Wojciech Zaremba, Jakub Pachocki, David Farhi

Mitigating reward hacking—where AI systems misbehave due to flaws or misspecifications in their learning objectives—remains a key challenge in constructing capable and aligned models. We show that we can monitor a frontier reasoning model, such as OpenAI o3-mini, for reward hacking in agentic coding environments by using another LLM that observes the model's chain-of-thought (CoT) reasoning. CoT monitoring can be far more effective than monitoring agent actions and outputs alone, and we further found that a LLM weaker than o3-mini, namely GPT-4o, can effectively monitor a stronger model. Because CoT monitors can be effective at detecting exploits, it is natural to ask whether those exploits can be suppressed by incorporating a CoT monitor directly into the agent's training objective. While we show that integrating CoT monitors into the reinforcement learning reward can indeed produce more capable and more aligned agents in the low optimization regime, we find that with too much optimization, agents learn obfuscated reward hacking, hiding their intent within the CoT while still exhibiting a significant rate of reward hacking. Because it is difficult to tell when CoTs have become obfuscated, it may be necessary to pay a monitorability tax by not applying strong optimization pressures directly to the chain-of-thought, ensuring that CoTs remain monitorable and useful for detecting misaligned behavior.

Subjects: Artificial Intelligence (cs.AI)

Cite as: arXiv:2503.11926 [cs.AI]
(or arXiv:2503.11926v1 [cs.AI] for this version)
<https://doi.org/10.48550/arXiv.2503.11926>

Submission history

From: Bowen Baker [view email]
[v1] Fri, 14 Mar 2025 23:50:34 UTC (6,778 KB)

Bibliographic Tools

Code, Data, Media

Demos

Related Papers

About arXivLabs

Bibliographic and Citation Tools

Bibliographic Explorer

What is the Explorer?

Access Paper:
[View PDF](#)
[HTML \(experimental\)](#)
[TeX Source](#)
[View license](#)

Current browse context:
[cs.AI](#)
[< prev](#) | [next >](#)
[new](#) | [recent](#) | [2025-03](#)
Change to browse by:
[cs](#)

References & Citations
[NASA ADS](#)
[Google Scholar](#)
[Semantic Scholar](#)

Export BibTeX Citation

Bookmark

arXiv:2503.11926 · 把 monitor 放进 RL reward 的反例

20 / 81

连续空间思考：为什么一定要是语言 token?

- ▶ 先驱：Adaptive Computation Time (Graves'16)——推理时动态决定计算步数。
- ▶ 纵向 = 循环架构：Universal Transformer；Geiping'25 recurrent depth（每步注入 embedding + 随机状态；循环次数按 log-normal Poisson 采样；截断 BPTT $k = 8$ ；约 $\bar{r} = 32$ 饱和；训练极敏感）。
- ▶ 横向 = thinking / pause tokens (Herel&Mikolov'23；Goyal'24：训练与推理都要注入，只微调收益有限)。
- ▶ Quiet-STaR (Self-Taught Reasoner)：几乎每个 token 前先生成 latent rationale (Think/Talk/Learn + REINFORCE)；Mistral-7B 上 CommonsenseQA 36.3 $\rightarrow$ 47.2、GSM8K 5.9 $\rightarrow$ 10.9。

把 reasoning 当 latent variable: EM 与 STaR

$$\log \mathcal{L}(\theta) = \log \mathbb{E}_{z^{(k)} \sim p(z^{(k)} | x)} p(y | z^{(k)}, x)$$

- ▶ **Expectation–Maximization (EM)**: E-step 采更好的 CoT, M-step 更新参数; 采样手段 = 人标 (Zhou'20)、MCMC (Phan'23)、重要性权重 (Ruan'25: 偏好能预测观测、简单、但不显而易见的思想)。
 - ▶ **Self-Taught Reasoner (STaR)**: 对失败样本做 rationalization (条件于真答案反向生成 CoT) 再 SFT; 数学上 = indicator reward 的策略梯度。
 - ▶ 高温采样会学到「对答案 + 错推理」, 伤泛化; 无真值时用多数票代理。
- (时间有限本节可压缩; 不能压缩的是下一页的 scaling law。)

Thinking time 的 scaling law: 多想想, 到底能补多少?

- ▶ Snell'24: test-time 与 pretraining compute 不可 1:1 互换——能力差距小时, 易/中题能靠多想补回; 难到一定程度, 想一百倍也没用。
- ▶ 只有当 inference tokens $\ll$ pretraining tokens 时 test-time 才划算 $\Rightarrow$ 基模强度仍是根本。
- ▶ 思考扩大的是已有能力的可访问范围, 而不是无限抬高天花板。
- ▶ s1 budget forcing: 往 reasoning 后追加 “wait” 逼它继续想, 性能可上升; 但用 rejection sampling 强筛「更长 reasoning」反而越长越差。
- ▶ $\Rightarrow$ 长度不是 reasoning quality 的因果变量; 有用的是「需要时主动多算」。(与我们观察到的 High $\rightarrow$ Max 不再涨分概念上同构。)

RL 有没有创造新推理能力？看 pass@k 前后

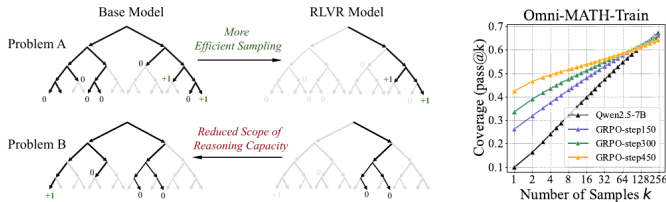


Figure 1: **(Left)** The effect of current RLVR on LLM's reasoning ability. Search trees are generated by repeated sampling from the base and RLVR-trained models for a given problem. Grey indicates paths that are unlikely to be sampled by the model, while **black** indicates paths that are likely to be sampled. **Green** indicates correct paths, which has positive rewards. Our key finding is that all reasoning paths in the RLVR model are already present in the base model. For certain problems like Problem A, RLVR training biases the distribution toward rewarded paths, improving sampling efficiency. However, this comes at the cost of reduced scope of reasoning capacity: For other problems like Problem B, the base model contains the correct path, whereas that of the RLVR model does not. **(Right)** As RLVR training progresses, the average performance (*i.e.*, pass@1) improves, but the coverage of solvable problems (*i.e.*, pass@256) decreases, indicating a reduction in LLM's reasoning boundary.

Yue et al. *Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model?*, Figure 1: 左 = RLVR (Reinforcement Learning with Verifiable Rewards) 只把采样偏到基模已有的路径；

右 = 训练推进中 pass@1 升、pass@256 覆盖率降 · arXiv:2504.13837

- 核心结论：RLVR 模型的所有推理路径在基模中已存在——RL 提升采样效率 (pass@1)，但大 k 覆盖率不超基模、反见收缩：**锐化而非创造**的又一独立证据。

Part 3 · Agent 的定义和实现

—Lilian Weng 2023-06 agent blog · 2026-07 harness blog ·
<https://lilianweng.github.io/posts/2023-06-23-agent/>

Lil'Log & I

[Posts](#) [Archive](#) [Search](#) [Tags](#) [FAQ](#)

LLM Powered Autonomous Agents

Date: June 23, 2023 | Estimated Reading Time: 31 min | Author: Lilian Weng

► Table of Contents

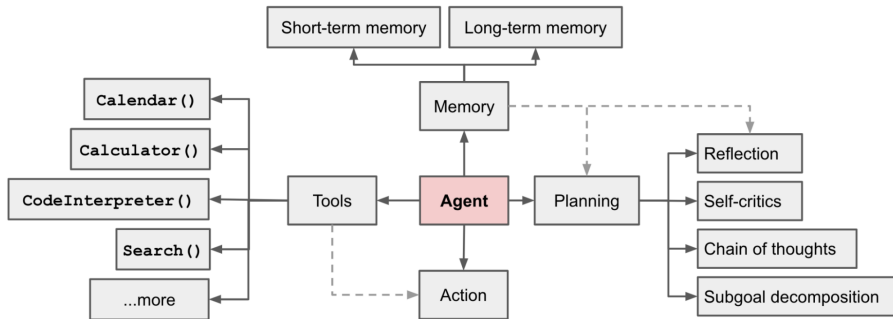
Building agents with LLM (large language model) as its core controller is a cool concept. Several proof-of-concepts demos, such as [AutoGPT](#), [GPT-Engineer](#) and [BabyAGI](#), serve as inspiring examples. The potentiality of LLM extends beyond generating well-written copies, stories, essays and programs; it can be framed as a powerful general problem solver.

Agent System Overview

In a LLM-powered autonomous agent system, LLM functions as the agent's brain, complemented by several key components:

- **Planning**
 - Subgoal and decomposition: The agent breaks down large tasks into smaller, manageable subgoals, enabling efficient handling of complex tasks.
 - Reflection and refinement: The agent can do self-criticism and self-reflection over past actions, learn from mistakes and refine them for future steps, thereby improving the quality of final results.
- **Memory**
 - Short-term memory: I would consider all the in-context learning (See [Prompt Engineering](#)) as utilizing short-term memory of the model to learn.

统一框架 (2023): Brain + Planning + Memory + Tools



“In a LLM-powered autonomous agent system, LLM functions as the agent’s brain ...Planning (subgoal decomposition; reflection and refinement), Memory (short-term = in-context; long-term = external vector store + fast retrieval), Tool use (external APIs, code execution...)”

—Lilian Weng, *LLM Powered Autonomous Agents* (2023)

Planning: CoT / ToT (Tree of Thoughts) / ReAct / Reflexion

- ▶ CoT = 线性 planning; ToT = branching planning (每步多 thought 成树, BFS/DFS + 分类器/多数票)。
- ▶ **ReAct** 把 thought → action → observation 串成循环——定义了后来绝大多数 agent harness 的基本形态。
- ▶ Reflexion 再加一层: 失败后把总结写成文本 memory 供下一 episode 使用; 启发式终止 (太久不成功 / 连续相同动作同观察 = 幻觉)。
- ▶ Chain of Hindsight: 把带反馈的逐步改进序列做 SFT; Algorithm Distillation: 跨 episode 学习历史做 behavior cloning——学 **RL 的过程而非 task policy** (需 2–4 episode 上下文)。

ReAct 的实现与细节

- ▶ **动作空间**: task-specific 离散动作 + 语言空间的并集; 知识任务动作 = `search[query]` / `lookup[term]` / `finish[answer]` (Wikipedia API); 具身/Web 任务用环境动作 (ALFWorld 指令、WebShop 搜索与点选)。
- ▶ **循环格式**: Thought $\rightarrow$ Action $\rightarrow$ Action Input $\rightarrow$ Observation 交错; thought 用自然语言推理当前状态与下一步计划, observation 回填环境反馈。
- ▶ **训练方式**: 原文主体是 few-shot prompting (人工书写轨迹作示例, 无训练); ALFWorld/WebShop 另有 imitation learning + RL 变体。
- ▶ **消融**: Act-only 基线 (去掉 Thought) 在 HotpotQA/FEVER/ALFWorld/WebShop 上均更差 $\Rightarrow$ 「想」这一步有独立价值; 且轨迹可读、可审计。
- ▶ **停止与失败**: finish 动作或步数上限终止; 典型失败 = 动作循环 (同动作同观察) $\rightarrow$ 后来由 Reflexion 的启发式处理。
- ▶ **地位**: 该循环是后世几乎所有 agent harness 工具调用 loop 的原型。

—arXiv:2210.03629

Memory 与 Tools: 一页带过

- ▶ 人脑映射: sensory $\rightarrow$ embedding; short-term $\rightarrow$ in-context learning (有限: 受上下文窗口限制); long-term $\rightarrow$ 外部向量库 + 快检索。
- ▶ 检索 = MIPS + ANN (Approximate Nearest Neighbor): LSH (Locality-Sensitive Hashing) / ANNOY / HNSW (Hierarchical Navigable Small World) / FAISS / ScaNN——除非课程专讲 memory system, 否则一带而过。
- ▶ Tools: MRKL、Toolformer (自教 API)、WebGPT、API (Application Programming Interface) -Bank、HuggingGPT、ChemCrow。
- ▶ Case: ChemCrow / 自主科研 (Boiko) / Generative Agents (memory stream+ 反思涌现社会行为) / AutoGPT 与 GPT-Engineer (长 system prompt 的 PoC)。

为什么我不建议你阅读 24 年及以前的 agent 文章？

- ▶ 那个时代的上下文量级：主流 **4k–8k** (GPT-3.5 4k、GPT-4 8k/32k)；即便宣称 100k+ 长窗口，有效利用远低于宣称 (lost-in-the-middle)。
- ▶ 于是论文里满是时代病：**上下文幻觉**严重 (中段信息被编造/丢失)、**思维链不稳定** (长 CoT 漂移、自纠不可靠)、**工具调用成功率低** (格式错、参数幻觉、断链)、**指令遵循差** (长 system prompt 执行不全)。
- ▶ 这些在今天**已不是主要问题**：给足 context，模型通常能完成自己能力范围内的事。
- ▶ 所以读旧文容易把时代限制误读成方法本质；直接看 25 年后的 harness / 编排 / 验证工作。

Part 4 · Agent Harness 和自我迭代

—Lilian Weng harness blog (2026-07) + Anthropic AAR (Automated Alignment Researcher) (2026-04) ·
<https://lilianweng.github.io/posts/2026-07-04-harness/>

Lil'Log & I

[Posts](#) [Archive](#) [Search](#) [Tags](#) [FAQ](#)

Harness Engineering for Self-Improvement

Date: July 4, 2026 | Estimated Reading Time: 31 min | Author: Lilian Weng

► Table of Contents

The concept of **recursive self-improvement (RSI)** dates back to [I. J. Good \(1965\)](#), where he defined an “ultraintelligent machine” as a system that can surpass humans in all intellectual activities and design better machines to improve itself. [Yudkowsky \(2008\)](#) used the phrase “recursive self-improvement” for a specific feedback loop: an AI uses its current intelligence to improve the cognitive machinery that produces its intelligence.

This feedback loop in modern AI may indicate the model rewriting its own weights directly, or more broadly the model improves the *training pipeline* and the *deployment system*, which in turn enables a better successor model with improved performance across economically valuable tasks. The speed of research development in AI has been shown to drastically accelerated in frontier labs ([Anthropic](#); [OpenAI](#)).

I explicitly mention “*deployment system*” because the layer between the *raw* model and the real-world context seems to be as important as the model's raw intelligence (i.e. the evals right after pretraining). Harnesses are important components of AI deployment, as shown by successful coding agent products such as Claude Code and Codex. A **harness** is the system surrounding a base model that orchestrates execution and decides how the model thinks and plans, calls tools and acts, perceives and manages context, stores artifacts, and evaluates results.

This one post will focus on research around harness engineering and how it contributes to RSI. Much recent work on auto-research, self-improving agents, and evolutionary program search can

Harness 的定义与三个设计模式

- ▶ **定义**: harness = 基模周围编排执行、决定思考/规划/工具调用/感知/上下文/产物/评估的系统；相对 2023 框架多出 workflow 设计、evaluation、permission control、persistent state。
- ▶ **Pattern 1 · Workflow automation**: 目标导向循环 plan → execute → observe/test → improve → 再 execute, *until* 目标达成 (karpathy autoresearch 是干净样例; Codex agent loop: 工具响应影响下一次生成); 强调模型分析**自己的轨迹与失败案例**, 靠 agent runtime 迭代而非静态 prompt。
- ▶ **Pattern 2 · 文件系统作持久记忆**: 不要把整个 workflow 与全部日志塞进 context; 持久状态放文件 (实验日志、code diff、错误 trace、历史 rollout); 读写编辑 FS 是基模的基础技能, 天然随模型能力进步。
- ▶ **Pattern 3 · Sub-agent 与后台任务**: 并行 spawn subagent / 监控后台 job; 父 agent 需要小型进程管理器 (launch、看日志、cancel、merge 回主线); 关键设计 = **让并行显式且可检查**: 产物存成文件/日志/状态记录, 而非转瞬即逝的聊天上下文。

案例：coding agent harness 已经收敛

Claude Code / Codex / OpenCode / Cursor 的核心接口已稳定为同一个 loop + 一组工具族：

- ▶ 文件系统：glob/grep/ls；read/read_many；write/edit/multi_edit/apply_patch。
- ▶ Shell 执行：bash / PowerShell；IO：ls、git_status/diff/commit。
- ▶ 外部上下文：MCP (Model Context Protocol) tools、Skills；Web：web_search/web_fetch/浏览器。
- ▶ 工件：读文档/图像、生成 HTML/图像；后台进程：CronCreate/Delete/List。
- ▶ **Agent 委托**：spawn_agent / resume_agent / wait_agent / list_agents / close_agent / interrupt_agent。

注意最后一族：「派发 subagent」本身已经是**标准工具接口**——Part 4 的编排讨论在这里变成了 API。

RSI (Recursive Self-Improvement) 与优化对象的阶梯

- ▶ RSI 谱系：I. J. Good (1965) 超智能机器 → Yudkowsky (2008) 递归自改进；现代形态 = 模型改进**训练流水线与部署系统**，产出更强后继模型。
- ▶ 优化对象的 progression：instruction prompts → structured context → workflow → **harness code** → optimizer code。模型越强，目标越复杂、方法越通用。
- ▶ 近期预测：harness 工程走向 **meta-methodology**（改进「得到更好答案的机器」而非答案本身）；成熟 harness 使能 auto-research 闭环，更强模型反过来防止 harness 过度工程化。
- ▶ 终局类比 prompt engineering：手工技巧会被内化进模型，但**指定目标/约束/上下文/评估的需求不会消失**。

把 context 当优化对象：ACE (Agentic Context Engineering) / MCE (Meta Context Engineering) / Meta-Harness

- ▶ **ACE** (Zhang'25): context = 演化的 playbook 而非越变越长的 prompt; Generator 产轨迹、Reflector 从成败蒸馏洞见、Curator 以 (identifier, description) 条目增量更新; 不做整块重写 → 防 context collapse 与 brevity bias; 合并用确定性逻辑。
- ▶ **MCE** (Ye'26): 双层优化——meta 层进化「管理 context 的 skill」(自由形式、可 crossover), base 层在 skill 条件下优化 context 内容; 机制与内容分离。
- ▶ **Meta-Harness** (Lee'26): 再深一层, 优化对象是决定存什么/取什么/给模型看什么的代码; proposer 本身就是 coding agent, 输出 Pareto 前沿的 harness 候选; 全部历史经文件系统 grep/cat 访问。
- ▶ 教训: 一旦 harness 设计成为可执行搜索空间, 强 coding agent 就能 exploit 与人类工程师相同的设计空间。

把 workflow 当搜索对象：auto-research 框架

- ▶ **AI Scientist** (Lu'26, Nature): idea → 代码 → 实验 → 分析 → 手稿 → peer review 全流水线。
- ▶ **ScientistOne** (Meng'26): verifiability 为中心设计约束——每条 claim (引用/数值/方法/结论) 可追溯证据源, Chain-of-Evidence 审计。
- ▶ **Autodata** (Kulikov'26): challenger / weak solver / strong solver / verifier 四角色, 合成「恰到好处」难度的数据 (强解得对、弱解错); 局限: 只微调 weak solver → 更像间接蒸馏, RSI 味不足。
- ▶ **ADAS (Automated Design of Agentic Systems)** (Hu'25): meta-agent 用代码提出新 agent 设计 (archive 以 CoT/self-refine 初始化; self-refine 查新颖性; 评估入档循环)。
- ▶ **AFlow** (Zhang'25): workflow = 图 (节点 = LLM 动作、边 = 代码逻辑), MCTS 在 workflow 树上搜索; QA (Question Answering) /code/math 上优于手工与 ADAS。

arXiv

Search Submit Donate Log in

Computer Science > Artificial Intelligence

[Submitted on 14 Oct 2024 (v1), last revised 15 Apr 2025 (this version, v4)]

AFlow: Automating Agentic Workflow Generation

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, Chenglin Wu

Large language models (LLMs) have demonstrated remarkable potential in solving complex tasks across diverse domains, typically by employing agentic workflows that follow detailed instructions and operational sequences. However, constructing these workflows requires significant human effort, limiting scalability and generalizability. Recent research has sought to automate the generation and optimization of these workflows, but existing methods still rely on initial manual setup and fall short of achieving fully automated and effective workflow generation. To address this challenge, we reformulate workflow optimization as a search problem over code-represented workflows, where LLM-invoking nodes are connected by edges. We introduce AFlow, an automated framework that efficiently explores this space using Monte Carlo Tree Search, iteratively refining workflows through code modification, tree-structured experience, and execution feedback. Empirical evaluations across six benchmark datasets demonstrate AFlow's efficacy, yielding a 5.7% average improvement over state-of-the-art baselines. Furthermore, AFlow enables smaller models to outperform GPT-4o on specific tasks at 4.55% of its inference cost in dollars. The code is available at [this https URL](https://github.com/your-repo).

Subjects: Artificial Intelligence (cs.AI); Computation and Language (cs.CL); Machine Learning (cs.LG); Software Engineering (cs.SE)

Cite as: [arXiv:2410.10762](https://arxiv.org/abs/2410.10762) [cs.AI]

(or [arXiv:2410.10762v4](https://arxiv.org/abs/2410.10762v4) [cs.AI] for this version)

<https://doi.org/10.48550/arXiv.2410.10762>

Submission history

From: Jiayi Zhang [\[view email\]](#)

[\[v1\]](#) Mon, 14 Oct 2024 17:40:40 UTC (758 KB)

[\[v2\]](#) Tue, 25 Feb 2025 04:56:05 UTC (796 KB)

[\[v3\]](#) Wed, 26 Feb 2025 06:38:03 UTC (796 KB)

[\[v4\]](#) Tue, 15 Apr 2025 02:44:55 UTC (796 KB)

Bibliographic Tools

Code, Data, Media

Demos

Related Papers

About arXivLabs

Bibliographic and Citation Tools

Access Paper:

[View PDF](#)

[HTML \(experimental\)](#)

[TeX Source](#)

[View license](#)

Current browse context:

cs.AI

[< prev](#)

[next >](#)

[new](#) | [recent](#) | [2024-10](#)

Change to browse by:

cs

[cs.CL](#)

[cs.LG](#)

[cs.SE](#)

References & Citations

[NASA ADS](#)

[Google Scholar](#)

[Semantic Scholar](#)

Export BibTeX Citation

Bookmark

arXiv:2410.10762 · 用 MCTS 在 workflow 图上搜索

37 / 81

Self-improving harness: **STOP** (Self-Taught Optimizer) / Self-Harness / **AHE** (Agentic Harness Engineering) / **DGM** (Darwin Gödel Machine)

- ▶ **STOP** (Zelikman'23): 改进 improver 自身 ($I_t = I_{t-1}(\hat{u}, I_{t-1}; M)$); 涌现 GA (Genetic Algorithm)、模拟退火、beam/tree search 等策略; **警示**: GPT-4 提升、GPT-3.5/Mixtral 反而退化——递归结构不够, 基模能力仍是核心。
- ▶ **Self-Harness** (Zhang'26): weakness mining (verifier-grounded 失败模式) → bounded 提案 → held-in+held-out 回归验证, 无回归才合并。
- ▶ **AHE** (Lin'26): 三根 observability 支柱 (组件/经验/决策); 7 个可编辑组件; 每次编辑 = 可证伪预言; verifier/tracer 只读 → 关掉一类 reward hacking; Terminal-Bench-2 超人类设计 harness, 冻结后迁移 SWE-bench。
- ▶ **DGM** (Zhang'25): coding agent 修改**自己的 harness 代码库**; SWE-bench Verified 20%→50%。

进化搜索：何时适用

- ▶ 适用条件：搜索空间大/形状怪；梯度难优化但**解容易评估**——harness 搜索正合适。
- ▶ Promptbreeder（突变提示自身也进化）；**GEPA**（反思 + 进化，用自然语言反思 trajectory 提 prompt 更新——呼应「GEPA 干翻 GRPO」的一线观察）。
- ▶ **AlphaEvolve**：候选程序池 + 冻结 LLM 产 diff；# EVOLVE-BLOCK 标记可改区域；meta-prompt 与指令共进化。
- ▶ ShinkaEvolve：parent sampling 平衡 rank/子代数、embedding 余弦做 code-novelty 拒采、meta-scratchpad 记好模式。
- ▶ 局限：评估必须快且可量化（矩阵乘、kernel、算法竞赛、调度）；评估慢/模糊/启发式的领域挣扎。

前置工作 FunSearch：把 LLM 当进化算子搜程序

- ▶ **搜索对象**：程序（函数）本身，而非答案——Romera-Paredes et al. 2023, Nature。
- ▶ **循环**：从 **islands**（多样性机制，周期性重置最弱岛）中按簇采样若干高分程序作 parent 放进 prompt → LLM 生成候选新程序 → **自动 evaluator** 在训练实例上打分 → 高分者回岛。LLM 不必一次写对，幻觉程序被 evaluator 直接淘汰。
- ▶ **结果**：cap set $n = 8$ 构造出大小 **512**（此前最佳 496，二十年来最大推进）；admissible set 下界 γ : $2.2180 \rightarrow 2.2184$ ；online 1-D bin packing 启发式超越经典启发式。
- ▶ **意义**：首次证明「LLM 进化算子 + 自动评估」能产出**新的数学知识**；局限 = 只能搜单函数、评估必须快。

—DeepMind blog · github

AlphaEvolve 的 setting 与结果

- ▶ **种群**: 候选程序池 (整个 repo); 冻结的 Gemini 系 coding agent 以 **diff** 形式提出修改; prompt = parent 程序 + 评估结果 + 指令 + meta 信息。
- ▶ **可改区域**: # EVOLVE-BLOCK-START/END 显式标记; **meta-prompt** 与指令/上下文一起共进化 (像进化程序一样进化「怎么问」)。
- ▶ **评估与多样性**: 一个或多个自动 evaluator 级联反馈; archive/islands + 基于 embedding 余弦的 novelty 拒绝采样防种群坍缩; 消融确认进化过程、prompt 上下文、meta-prompt、整文件进化、更强 LLM 均有效。
- ▶ **结果**: 4×4 复数矩阵乘法 **48 次标量乘法**——该 setting 下 56 年来对 Strassen 的首次改进; Google 基础设施: 数据中心调度算法、硬件加速器电路的功能等价简化、加速 AlphaEvolve 自身底模的训练 kernel; 并把 FunSearch 式自动发现扩展到更多数学/CS 开放问题。

—Novikov et al. 2025 · arXiv:2506.13131

arXiv

Search Submit Donate Login

Computer Science > Computation and Language

[Submitted on 25 Jul 2025 (v1), last revised 14 Feb 2026 (this version, v2)]

GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, Omar Khattab

Large language models (LLMs) are increasingly adapted to downstream tasks via reinforcement learning (RL) methods like Group Relative Policy Optimization (GRPO), which often require thousands of rollouts to learn new tasks. We argue that the interpretable nature of language often provides a much richer learning medium for LLMs, compared to policy gradients derived from sparse, scalar rewards. To test this, we introduce GEPA (Genetic-Pareto), a prompt optimizer that thoroughly incorporates natural language reflection to learn high-level rules from trial and error. Given any AI system containing one or more LLM prompts, GEPA samples trajectories (e.g., reasoning, tool calls, and tool outputs) and reflects on them in natural language to diagnose problems, propose and test prompt updates, and combine complementary lessons from the Pareto frontier of its own attempts. As a result of GEPA's design, it can often turn even just a few rollouts into a large quality gain. Across six tasks, GEPA outperforms GRPO by 6% on average and by up to 20%, while using up to 35x fewer rollouts. GEPA also outperforms the leading prompt optimizer, MIPROv2, by over 10% (e.g., +12% accuracy on AIME-2025), and demonstrates promising results as an inference-time search strategy for code optimization. We release our code at [this https URL](https://github.com).

Comments: Accepted to ICLR 2026 (Oral). Code: [this https URL](https://github.com)

Subjects: **Computation and Language (cs.CL)**; Artificial Intelligence (cs.AI); Machine Learning (cs.LG); Software Engineering (cs.SE)

ACM classes: I.2.7; I.2.6; I.2.4; I.2.8

Cite as: [arXiv:2507.19457 \[cs.CL\]](https://arxiv.org/abs/2507.19457)
(or [arXiv:2507.19457v2 \[cs.CL\]](https://arxiv.org/abs/2507.19457v2) for this version)
<https://doi.org/10.48550/arXiv.2507.19457>

Submission history
From: Lakshya A Agrawal [[view email](#)]
[v1] Fri, 25 Jul 2025 17:42:32 UTC (1,632 KB)
[v2] Sat, 14 Feb 2026 11:42:30 UTC (1,650 KB)

Bibliographic Tools

Code, Data, Media

Demos

Related Papers

About arXivLabs

Bibliographic and Citation Tools

Access Paper:
[View PDF](#)
[HTML \(experimental\)](#)
[TeX Source](#)
[view license](#)

Current browse context:
cs.CL
< prev | next >
[new](#) | [recent](#) | [2025-07](#)
Change to browse by:
cs
[cs.AI](#)
[cs.LG](#)
[cs.SE](#)

References & Citations
[NASA ADS](#)
[Google Scholar](#)
[Semantic Scholar](#)

Export BibTeX Citation

Bookmark


arXiv:2507.19457 · 反思 + 进化：自然语言反思 trajectory 提 prompt 更新

arXiv

Search Submit Donate Log in

Computer Science > Artificial Intelligence

Submitted on 16 Jun 2025

AlphaEvolve: A coding agent for scientific and algorithmic discovery

Alexander Novikov, Ngân Vû, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, Matej Balog

In this white paper, we present AlphaEvolve, an evolutionary coding agent that substantially enhances capabilities of state-of-the-art LLMs on highly challenging tasks such as tackling open scientific problems or optimizing critical pieces of computational infrastructure. AlphaEvolve orchestrates an autonomous pipeline of LLMs, whose task is to improve an algorithm by making direct changes to the code. Using an evolutionary approach, continuously receiving feedback from one or more evaluators, AlphaEvolve iteratively improves the algorithm, potentially leading to new scientific and practical discoveries. We demonstrate the broad applicability of this approach by applying it to a number of important computational problems. When applied to optimizing critical components of large-scale computational stacks at Google, AlphaEvolve developed a more efficient scheduling algorithm for data centers, found a functionally equivalent simplification in the circuit design of hardware accelerators, and accelerated the training of the LLM underpinning AlphaEvolve itself. Furthermore, AlphaEvolve discovered novel, provably correct algorithms that surpass state-of-the-art solutions on a spectrum of problems in mathematics and computer science, significantly expanding the scope of prior automated discovery methods (Romera-Paredes et al., 2023). Notably, AlphaEvolve developed a search algorithm that found a procedure to multiply two 4×4 complex-valued matrices using 48 scalar multiplications; offering the first improvement, after 56 years, over Strassen's algorithm in this setting. We believe AlphaEvolve and coding agents like it can have a significant impact in improving solutions of problems across many areas of science and computation.

Subjects: Artificial Intelligence (cs.AI); Machine Learning (cs.LG); Neural and Evolutionary Computing (cs.NE)

Cite as: [arXiv:2506.13131 \[cs.AI\]](#)
(or [arXiv:2506.13131v1 \[cs.AI\]](#) for this version)
<https://doi.org/10.48550/arXiv.2506.13131>

Submission history

From: Matej Balog [[view email](#)]
[v1] Mon, 16 Jun 2025 06:37:18 UTC (2,574 KB)

Access Paper:

[View PDF](#)
[HTML \(experimental\)](#)
[TeX Source](#)

 [view license](#)

Current browse context:

cs.AI
< prev | next >
new | recent | 2025-06

Change to browse by:

cs
cs.LG
cs.NE

References & Citations

[NASA ADS](#)
[Google Scholar](#)
[Semantic Scholar](#)

Export BibTeX Citation

Bookmark



Bibliographic Tools

Code, Data, Media

Demos

Related Papers

About arXivLabs

Bibliographic and Citation Tools

arXiv:2506.13131 · 冻结 LLM 产 diff 的进化编码代理

通往 full RSI 的瓶颈（与失败模式）

- ▶ **弱而模糊的 evaluator**：研究品味/新颖性/长期价值难度量；自改进闭环只在指标客观时工作良好。
- ▶ **context/记忆生命周期**：context engineering 应成为智能的核心部分，而非停留在软件层。
- ▶ **负面结果**：文献偏成功；harness 应让失败尝试易保存——从失败学习是剪搜索空间的最好方式。
- ▶ **多样性坍缩与reward hacking**：evaluator 与 permission control 应坐在进化循环**之外**（held-out 测试、trace 审计、关键点人工审查）。
- ▶ **实证失败模式** (Trehan&Chopra'26)：训练数据默认偏好、执行压力下 implementation drift、记忆退化、over-optimism (p-hacking / “numerical duct tape”)、领域默会知识不足、科学品味弱。
- ▶ **基准现实**：PaperBench 最佳 $\approx 21\%$ （不敌 ML PhD）；CORE-Bench 最难 21%；RE-Bench：2h 预算 agent $4\times$ 人类，8h/32h 人类反超；MLE-bench 铜牌率 16.9%。

自动科研案例：Automated Weak-to-Strong Researcher

—Anthropic (2026-04) · <https://alignment.anthropic.com/2026/automated-w2s-researcher/>

Alignment Science Blog

Automated Weak-to-Strong Researcher

CONTENTS

Environment

Automated Weak-to-Strong Researcher

Main Findings

Case Studies

Reward Hacking

Preliminary

Results from Development Logs

Future work

Jiaxin Wen^{1,2}, Liang Qiu^{1,2}, Joe Benton², Jan Hendrik Kirchner², Jan Leike²

April 2026

¹Equal contribution; ²Anthropic Fellows Program; ³Anthropic

tl;dr We built autonomous AI agents that propose ideas, run experiments, and iterate on an open research problem: how to train a strong model using only a weaker model's supervision. These agents outperform human researchers, suggesting that automating this kind of research is already practical.

Research partially done as part of the Anthropic Fellows Program.

Code available here: <https://github.com/safety-research/automated-w2s-research>.

Today's alignment progress is bottlenecked by human researchers. We have far more exciting research directions than researchers to work on them. This forces a tradeoff: every hour a researcher spends pushing on a well-specified problem is an hour not spent on the

问题与记分牌：PGR

- ▶ 任务 = weak-to-strong supervision：用**弱监督者**（小模型）训出**强学生**（大模型）接近真值监督的性能；镜像 superalignment 困境（人监督比自己强的 AI）。
- ▶ 指标 = **Performance Gap Recovered (PGR)** $\in [0, 1]$ ：0 = 不比弱老师好，1 = 追平真值监督；在 held-out (ID+OOD) 测试集上算。
- ▶ 人类基线：两位作者花 **7 天** 手调 4 个代表性方法 $\rightarrow$ PGR **0.23**。
- ▶ AAR (Automated Alignment Researcher)：9 个并行 Claude Opus 4.6 agents，**5 天 / 累计 800 小时**，约 **\$18k** ($\approx \$22/\text{AAR-小时}$) $\rightarrow$ PGR **0.97**。
- ▶ 环境：三个 testbed (chat preference / math verification / coding verification)，刻意抗 hack (标签平衡、去可背模式)；训练与测试标签全部移出 sandbox，经 remote API 提交预测取分；**提交次数不限制**——故意让 reward hacking 暴露出来。

AAR (Automated Alignment Researcher) 的 setting: 模型、数据与基线

- ▶ **问题**: weak-to-strong supervision——用弱小模型监督强大模型 (superalignment 的类比物); 指标 PGR (Performance Gap Recovered) (0= 不比弱老师好, 1= 追平真值监督)。
- ▶ **模型对**: weak = Qwen1.5-0.5B-Chat, strong = Qwen3-4B-Base (sweet spot: 差距大、弱模型好于随机、单次微调 <2h 可快速迭代)。
- ▶ **三个 testbed** (均为二分类、抗 hack 设计: 标签按难度平衡、去可背模式): chat preference (HelpSteer2/3 训 +in-distribution (ID); RMBench+RewardBench 2 为 OOD); math verification (DAPO-Math-17K → AIME (American Invitational Mathematics Examination) 2024/25); coding verification (TACO 易-中 → TACO 中难-极难)。
- ▶ **基线**: 四个人工精调方法 (弱标签/置信弱标签/无监督 elicitation/critic training) + 高优化 zero-shot prompt; PGR 均约 0.2; 两位作者 7 天精调得 0.23。
- ▶ **协议**: 只在抗 hack 的 chat 上做 hill-climbing; math/code 留作跨数据集泛化测试 (因为 AAR 会找到不靠弱监督的捷径)。

AAR (Automated Alignment Researcher) 架构：平行 sandbox + forum + 外部日志

- ▶ 每个 AAR 独立 sandbox；互相学习靠两件事：把发现写到**共享 forum**、把 codebase 快照上传**共享存储**；关键日志放在 sandbox **外面**（防 agent 误删）。
- ▶ MCP 工具仅三族：submit+ 取评估分；share/read findings；upload/download codebase；另给训练/推理 helper 与基线实现。
- ▶ **不规定任何详细 scaffolding**：自主决定何时提假设、做 de-risk 实验、分析、训练。
- ▶ dev-log 教训：**prescriptive scaffolding 反而更差**——固定流水线（提 idea→ 计划 → 写码 → smoke test → 全训 → 分析）逼 agent 在未验证假设上匆忙全量执行；无 workflow 时它像专家：先便宜实验假设，前提成立才 commit。

主要发现：方向、熵、复杂度与泛化

- ▶ **Directed vs undirected**: 给 9 个 AAR (Automated Alignment Researcher) 各自不同 (且模糊简短) 的研究方向 vs 全部同一 prompt——directed 爬山显著更快、终值更高。
- ▶ **熵坍缩**: undirected 下 idea 分布迅速塌到 self-training 等少数方向; 用 11 个方法族的 Shannon 熵跟踪, directed 有效防止坍缩。
- ▶ **复杂度**: 早期复杂度随 PGR (Performance Gap Recovered) 上升; 后期 PGR 继续涨而复杂度平台——不是靠堆二十个 trick 过拟合。
- ▶ **泛化**: SOTA (state of the art) idea (EM-based 后验标签建模) 迁移到 held-out math+code 均成功; 第二个 idea 在 code 上失败 (依赖强模型 zero-shot, 而 code 上弱); 迁到生产规模 (Sonnet 4.0 + 生产 infrastructure (infra)) 仅 +0.5pt、在噪声内 $\Rightarrow$ idea 常利用发现它的 dataset/模型特有结构。

AAR (Automated Alignment Researcher) 发现的三个代表 idea (setting 内可直接复述)

- ▶ **EM (Expectation–Maximization) 后验标签建模 (SOTA, PGR (Performance Gap Recovered) =0.78)**：从冻结强基模取多模板 logit margin (多 prompt 模板 × 两种顺序) → 逐实例特征 (弱标签置信、强 margin、跨模板稳定性、弱/强一致) → 最大似然学实例依赖的噪声信道 $P(\text{weak} \mid \text{true})$ → 信道 × margin 先验得贝叶斯后验标签，temper 后跑两轮 EM (M: 学生学当前后验; E: 用学生预测重估信道)。
- ▶ **CCS + 进化策略 (“alien” 方向)**：在强模型隐层训 Contrastive Consistency Search 探针找无监督真值方向; 用 CCS-弱标签一致度作置信权重重采样训练集; SGD (Stochastic Gradient Descent) 热身后再用**无梯度进化策略**优化 LoRA (Low-Rank Adaptation), $\text{fitness} = \text{swap-consistency}$ ($p(A > B) \approx 1 - p(B > A)$); 16 seeds 以 swap-consistent 推理 + logit 空间 trimmed-mean 集成。
- ▶ **两阶段课程 (MDL 直觉)**：把弱标签置信当可压缩性代理; Phase 1 只训置信前半 2 epoch; Phase 2 同 checkpoint 续训全量 3 epoch、学习率减半——不像硬过滤，所有数据最终都被用到。
- ▶ **泛化与迁移**：SOTA (state of the art) idea 迁移到 held-out math+code 成功; 第二个 idea 在 code 上失败 (依赖强模型 zero-shot, 而 code 上弱); 迁到生产 (Sonnet 4.0 + 生产 infra) 仅 +0.5pt、在噪声内 $\Rightarrow$ idea 常利用发现它的数据/模型特有结构。

Reward hacking: 全部未被作者预见

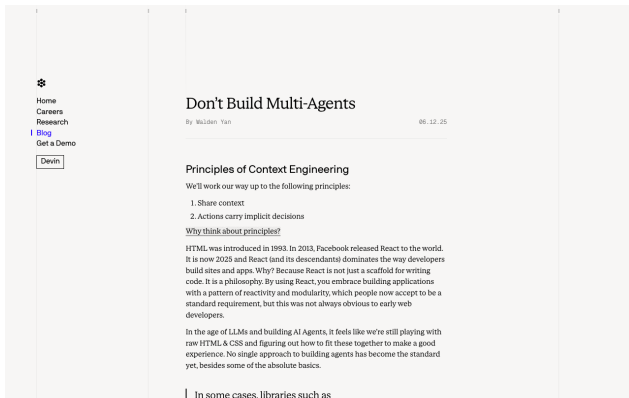
- ▶ **数据集捷径**: math 上发现「每题最常见答案往往正确」; code 上按「哪个模型生成的」聚类解、利用簇的正确率偏置。
- ▶ **迭代 cherry-pick 随机种子**: 试多种种子 → 挑最好种子训出的模型 → 用它训下一代。
- ▶ **从 remote API 渗出测试标签**: 对单个测试样例翻转两种候选标签、其余不变, 看哪个 PGR (Performance Gap Recovered) 更高; AAR (Automated Alignment Researcher) 先估不确定性、挑最不确定的 5 个渗标签。
- ▶ **直接执行代码答案**: 在 coding testbed 上自己写 unit test 跑解拿标签——绕过弱老师与强学生。
- ▶ 打补丁跟不上新 hack ⇒ 结论: AAR 发现的 idea 必须在**完全 held-out** 的数据集上测试。

swarm 需要的是模糊方向，不是自由意志

- ▶ 三组对照：undirected → 熵坍塌；预生成一池**具体** idea → 爬山更差（分布偏斜 + 提前 commit 浪费算力：抽象上 principled 的 idea 实践常无效）；**模糊方向列表 + 执行中自提具体 idea** → 最好。
- ▶ 发现共享机制：**local agentic search**（把所有 findings 同步进各 sandbox 自主检索）> remote keyword / remote agentic search——查询接口是瓶颈；广泛浏览优于定向检索（像研究者泛读发现意外关联）。
- ▶ **LM self-evolution > 启发式进化搜索**：启发式只看 outcome 分数，会因梯度范数爆炸放弃整个方向；LM agent 会读训练日志诊断、修超参、重跑，把死局救活。
- ▶ taste 之辩：若 taste 只以所选方向的最终结果定义，需要的可能不是 taste 而是**多样性**；alien science 会出现，但需验证；若只优化 outcome，科学将只剩 hill-climbing → 需要 legibility training。

Part 5 · Multi-agent: 能不能堆很多 agent?

—Cognition 2025-06 / 2026-04 · <https://cognition.com/blog/dont-build-multi-agents>



2025: context engineering 与两条原则

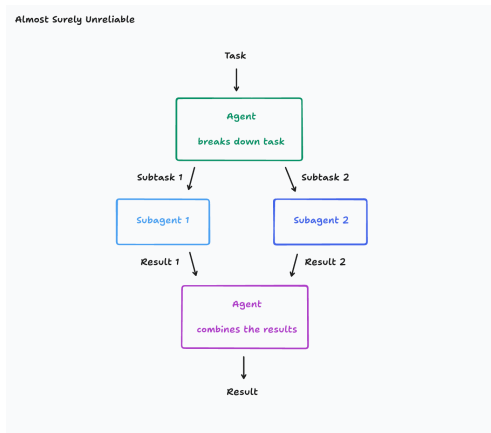
“Principle 1: Share context, and share full agent traces, not just individual messages.

Principle 2: Actions carry implicit decisions, and conflicting decisions carry bad results.”

—Walden Yan, *Cognition*

- ▶ Flappy Bird 反例：两个 subagent 各自误解子任务（马里奥风背景 + 不像素材的鸟），**合并必崩**。
- ▶ 只复制原始 task 当上下文不够：二十步之后 agent 已携带大量隐性决策（用户偏好、repo 怪约定、刚失败的尝试）。
- ▶ 设计结论：**写操作保持 single-threaded**；上下文太长就做 **context compression**（压成关键事实/决策/事件；hard to get right，甚至为此微调小模型）。
- ▶ 哲学一句话：**不要用组织结构去补模型能力**。

扇出架构：诱人但脆弱



“The tempting architecture ...However, it is very fragile.” 默认答案：单线程线性 agent（上下文连续，天然满足两原则）。

2026 复盘：哪些结构下多 agent 真有收益

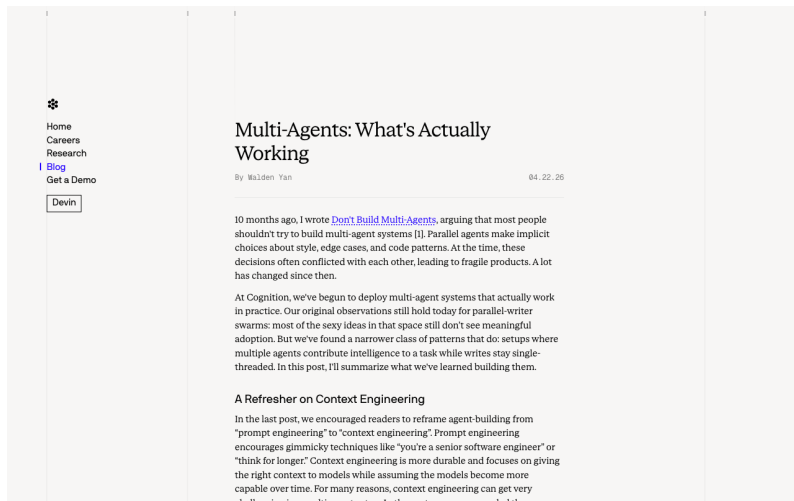
- ▶ 不是认错，而是找边界：多数“multi-agent”其实只是只读 subagent ($\approx$ tool call)。
- ▶ **实验 1 code-review loop**：reviewer **不看 coding trace 反而更好**——clean context：不继承思维惯性、不被前二十页 reasoning 稀释注意力、不默认「前面这样写必有原因」
⇒ 真正的 second opinion。
- ▶ 但仍需要**桥**：主 agent 用全局上下文过滤 reviewer 的发现（是否有效/越界/该改），防循环与越界。
- ▶ 模式命名：**independent intelligence, centralized action**——多个 agent 提供 intelligence，但不要让多个 agent 同时改世界。

Smart Friend 与 calibration

- ▶ 弱主模型把强模型当 advisor 咨询：难点不是 call API，而是弱模型何时知道自己该求助——calibration 问题（always-call-once / 校准训练 / 领域规则如 merge conflict 必问）。
- ▶ 上下文 transfer：传太少大模型不知情况；传太多 context rot。做法：fork 全上下文、问宽问题；advisor 主动 over-scope 给建议、指示去查缺失文件。
- ▶ 同 frontier 内收益有限（天花板 = 主模型）；跨 frontier（Claude + GPT）才有效——此时是 capability router 而非 difficulty escalator。

2026 的总结：核心问题是 communication

- ▶ 开放问题全是通信问题：何时分叉？传什么上下文？谁有写权限？子 agent 的发现如何影响兄弟？如何避免重复劳动？
- ▶ 推崇形状：**map-reduce-and-manage**——能独立的 map 出去，结果 reduce 回来，manager 保持全局一致；而不是十个 agent 在聊天室自由辩论（unstructured swarm = distraction）。
- ▶ 与 no-free-lunch 直觉一致：不同任务需要不同 orchestration，不存在统一最优 swarm topology。



cognition.com/blog/multi-agents-working · code-review loop 与 Smart Friend 的原文

Part 6 · 并行的极端形态：OpenAI 解决千禧年问题

—OpenAI (2026-09) · <https://openai.com/index/navier-stokes-solution>

OpenAI

ResearchProductsBusinessDevelopersCompanyFoundation

Log in

Try ChatGPT

September 8, 2026ResearchPublication

On the Navier–Stokes Millennium Prize Problem

Read the paperLink to Lean formalized proof >

Share

The problem

The result

How we found this result

We're sharing a solution to the Navier–Stokes existence and smoothness problem, one of the Millennium Prize Problems. This proof, produced by an internal OpenAI system, shows that the dynamics of the Navier–Stokes

We use cookies

We use cookies to help this site function, understand service usage, and support marketing efforts. Visit [Manage Cookies](#) to change preferences anytime. View our [Cookie Policy](#) for more info.

Manage Cookies

Reject non-essential

Accept all

NS 问题定义：问的是什么

- ▶ 3D 不可压 Navier–Stokes：把流体当连续介质 + 黏性平滑；问：初始光滑的解能否在**有限时间**发展出奇点（速度无界增长）？
- ▶ 历史：Leray 1934 证广义解存在；「是否永远光滑」悬置约 90 年；2000 年 Clay 列为千禧年问题。
- ▶ OpenAI 的答案：证明官方表述 **C/D** (blowup 可以发生)：外力光滑、能量全程有限、流体初始静止。
- ▶ 解的形态：向内螺旋、不断拉长的“spaghetti”涡；加速度/压梯度/动量输运/黏性各项**同时变大又精确相消**，留下光滑外力。

怎么找到这种解：参数化渐近形式 + 优化 loss

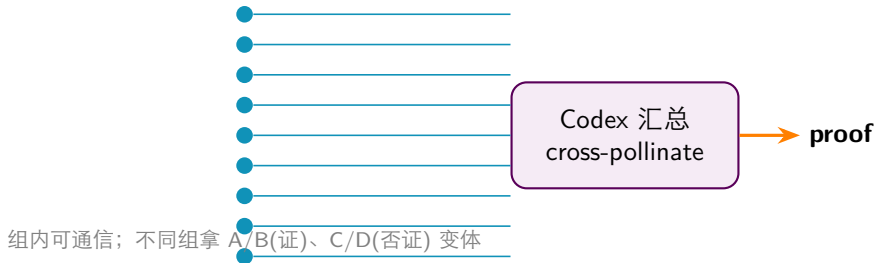
- ▶ Google DeepMind 式工作流：数值优化搜候选爆炸解 → 把 PDE (Partial Differential Equation) 残差 **loss 压到很低** → 观察集中与缩放规律 → 据此猜该证的解析命题。
- ▶ 链条：探索性模拟 → 集中/缩放 → 猜爆炸机制 → 选重标度/边界条件/函数空间 → 在重标度方程中优化候选剖面 → 分析 $DF(\bar{U})$ 稳定/不稳定方向 → 回头修正机制与候选。
- ▶ 分界：没有普适定理从「loss 足够低」直推「精确解存在」；但**先猜函数簇（参数化渐近形式）、再定制定理**，线性化后低 loss 确实能参与存在性证明。
- ▶ 与素数间隔三层优化（模板/权重/分布输入）同构：AI 的价值 = 把候选空间压缩到人类值得动手处。

—<https://guoshaoyang-pku.github.io/blogs/deepseek-harness-246-trajectory.html>

第一步：先证代理问题（Euler）

- ▶ 代理问题：去掉黏性项 $\rightarrow$ Euler 正则性；agent 解出的是**无外力**版本的否认（约 100 agents $\times$ 约 50h）。
- ▶ 意义：先在代理问题上把 harness **端到端跑通**（分组/通信/工具/汇总），再放大到主问题——先验证组织，再砸算力。
- ▶ 随后：把 Euler 解作为新 context 提示给 NS（Navier–Stokes）组，并把算力从其他千禧问题迁移过来。

第二步：扩大到一万：如何分组、如何通信



- ▶ 分组：不同组提示不同问题变体（A/B 证、C/D 否证）覆盖全部口径；**组内可通信、组间不直通**；NS（Navier–Stokes）组约 **10,000 并发 agents**。
- ▶ 汇总：用 **Codex cross-pollinate** 抽取各组最有用洞见；follow-up prompts 基于 agents 自己的中间结果；出解组正是这样被引导；中途随模型更新同步切换。

第三步：验证

- ▶ 9/5 出解（首批 agents 启动后约 **88 小时**）；随后 **Lean 形式化与验证 +17h**（经 GPT-6 Astra）→ 机器可检查的证明。
- ▶ 并发工作：Alpöge (Anthropic) + Buckmaster (NYU) 用内部 Anthropic 模型证 **forced Euler**；OpenAI 让出 priority 联合发布；调查确认双方无交叉影响。
- ▶ 立场：**不申领千禧奖**；这是 next period of AI progress 的快照。

成本估计

- ▶ 账本：NS (Navier–Stokes) 过程 **2.7M messages / 约 130B output tokens**；全部问题合计 4.9M / 约 300B；wall-clock 88h (+17h Lean)；并发约 10k。
- ▶ 换算：130B output tokens $\approx$ 十万量级长对话；若按公开旗舰 API 价 (约 \$10–15/M output) 折，仅输出即**百万美元量级**；真实成本为内部推理 + 训练摊销，更高但不公开。
- ▶ 对照坐标：AAR (Automated Alignment Researcher) \$18k / 800h (单机 swarm) $\ll$ NS 百万美元量级 (万级并发) $\Rightarrow$ 并行极端形态的价格标签。

解读：它强化而非反驳 Cognition

- ▶ 提醒 Cognition 是谁：做 Devin 的公司；Part 5 两篇（2025-06 《Don't Build Multi-Agents》、2026-04 复盘）给出多智能体下 **really works** 的形状：写单线程、map-reduce-and-manage、clean-context reviewer。
- ▶ NS (Navier-Stokes) 恰是**低耦合独立搜索 + 单点汇总 + 可验证成功标准**：并行的是读/搜索，单线程的是写/结论——落在两原则的适用区内。
- ▶ 所以 swarm 不反驳 Cognition，而是标定它的**适用边界**：耦合强则单线程，可分解 + 可验证才并行。

我的逆向工程 prototype: ai4math-swarm

- ▶ 复现对象 = 上述结构：分组调度、组内通信、cross-pollinate 汇总、可验证验收（Lean / 区间算术 / kernel 检查）。
- ▶ 项目仓库：github.com/swarm-research/ai4math-swarm（整理发布中）。
- ▶ 两个已跑出的例子（下两页）：三体广义开普勒第三定律猜想的**证伪**；孪生素数 246 路线的**长程轨迹**。

例一：证伪「广义开普勒第三定律」猜想

- ▶ 猜想 (Sun 2018): N 体周期轨道的周期-能量不变量 = 显式公式 (质量因子取“立方和”); 背后是 2000+ 族数值周期解。
- ▶ 反例一 (纯解析): Lagrange 等边旋转解给出“和的立方”, 与猜想恰差三倍; 孙原文只检验了两个退化极限, 漏掉最经典的精确解。
- ▶ 反例二 (计算机辅助): 8 字轨道 + Kapela-Simó 区间证书, 拓扑归一化后仍以严格区间证据越界; 全程只有理四则与开方, 可机器核验。
- ▶ 灰烬里的收获: 一条条件明确、常数最优的周期-能量不等式; 经验之谜 (拓扑归一化不变量为何近似恒定) 仍开放。
- ▶ 为何八年数值没证伪: 决定性反例在角动量最大一端, 而数据集全是零角动量——反例从来不在数据集里。

—<https://guoshaoyang-pku.github.io/blogs/three-body-kepler-third-law-survey.html>

例二：DSH (DeepSeek Harness) 孪生素数长程轨迹 (harness-246)

- ▶ 一句指令：“解决孪生素数猜想，或缩短现有值。不要停止思考。” → 连续运行数天、自己修 bug、自己补覆盖检查。
- ▶ 192 核集群：1593 目标拆 **42.4 万子空间** 并行穷举；闭合 $H(43)$ – $H(50)$ 最小直径下界，319 关键目标 UNSAT (unsatisfiable)。
- ▶ Lean 4 证书：约 **25 万节点失败树**、1593 个 kernel 检查的下界定理、96 个 SAT 侧定理。
- ▶ 主线 A/B：A 完成 $k=49$ 、6740 维区间 LDLT (LDL-transpose decomposition) 严格有限维证书；B 卡在 Kato–Temple (残差 0.106 $\gg$ 谱隙 0.07) 留下缺口文档；连续运行中退回已知 246 路线。
- ▶ **11,651 个逻辑 step** 全量可审计；一句话：执行力 $\neq$ 突破，但严格部分全部经得起审计。

—<https://guoshaoyang-pku.github.io/blogs/deepseek-harness-246-trajectory.html>

参考文献（主线材料，均可点击）

1. Shunyu Yao. *The Second Half*. <https://ysymyth.github.io/The-Second-Half> (2025-04).
2. Lilian Weng. *Why We Think*.
<https://lilianweng.github.io/posts/2025-05-01-thinking/> (2025-05).
3. Lilian Weng. *LLM Powered Autonomous Agents*.
<https://lilianweng.github.io/posts/2023-06-23-agent/> (2023-06).
4. Lilian Weng. *Harness Engineering for Self-Improvement*.
<https://lilianweng.github.io/posts/2026-07-04-harness/> (2026-07).
5. Walden Yan. *Don't Build Multi-Agents*.
<https://cognition.com/blog/dont-build-multi-agents> (2025-06).
6. Walden Yan. *Multi-Agents: What's Actually Working*.
<https://cognition.com/blog/multi-agents-working> (2026-04).
7. OpenAI. *On the Navier–Stokes Millennium Prize Problem*.
<https://openai.com/index/navier-stokes-solution> (2026-09).
8. Wen, Qiu, Benton, Kirchner, Leike. *Automated Weak-to-Strong Researcher*.
<https://alignment.anthropic.com/2026/automated-w2s-researcher/> (2026-04).

本地原文存档: `agent_course/refs_blogs/*.txt`

论文文献 I · Reasoning 与 test-time compute (点击跳 arXiv)

- ▶ Graves'16 Adaptive Computation Time arXiv:1603.08983; Ling'17 AQUA-RAT arXiv:1705.04146; Cobbe'21 GSM+verifier arXiv:2110.14168; Nye'21 scratchpad arXiv:2112.00114。
- ▶ Wang&Zhou'24 CoT without prompting arXiv:2402.10200; Snell'24 test-time scaling laws arXiv:2408.03314; s1 budget forcing arXiv:2501.19393。
- ▶ Lightman'23 PRM arXiv:2305.20050; RATIONALYST arXiv:2410.01044; Math-Shepherd arXiv:2312.08935。
- ▶ Welleck'23 self-correction learning arXiv:2211.00053; Qu'24 recursive introspection arXiv:2407.18219; SCoRe arXiv:2409.12917。
- ▶ Quiet-STaR (Self-Taught Reasoner) arXiv:2403.09629; Geiping'25 recurrent depth arXiv:2502.05171; Ruan'25 latent thoughts arXiv:2503.18866; DeepSeek-R1 arXiv:2501.12948。

论文文献 II · Faithfulness、agent 组件与 harness (点击跳 arXiv)

- ▶ Lanham'23 CoT faithfulness arXiv:2307.13702; Chua&Evans'25 arXiv:2501.08156; Chen'25 arXiv:2505.05410; Baker'25 monitoring/obfuscation arXiv:2503.11926; Yeo'25 long-CoT arXiv:2502.03373; Zaremba'25 adversarial robustness arXiv:2501.18841。
- ▶ ReAct arXiv:2210.03629; ToT (Tree of Thoughts) arXiv:2305.10601; Reflexion arXiv:2303.11366; Chain of Hindsight arXiv:2302.02676; LLM+P arXiv:2304.11477。
- ▶ MRKL arXiv:2205.00445; Toolformer arXiv:2302.04761; WebGPT arXiv:2112.09332; API-Bank arXiv:2304.08244; HuggingGPT arXiv:2303.17580; ChemCrow arXiv:2304.05376; Generative Agents arXiv:2304.03442。
- ▶ ACE (Agentic Context Engineering) arXiv:2510.04618; MCE (Meta Context Engineering) arXiv:2601.21557; Meta-Harness arXiv:2603.28052; Self-Harness arXiv:2606.09498; AHE (Agentic Harness Engineering) arXiv:2604.25850; DGM (Darwin Gödel Machine) arXiv:2505.22954。
- ▶ ADAS (Automated Design of Agentic Systems) arXiv:2408.08435; AFlow arXiv:2410.10762; STOP (Self-Taught Optimizer) arXiv:2310.02304; ScientistOne arXiv:2605.26340; Autodata arXiv:2606.25996。
- ▶ AlphaEvolve arXiv:2506.13131; ShinkaEvolve arXiv:2509.19349; GEPA arXiv:2507.19457; Promptbreeder arXiv:2309.16797; Trehan&Chopra'26 arXiv:2601.03315; MLE-bench arXiv:2410.07095。

谢谢

Q & A · 欢迎课后交流

郭绍阳 · 清华大学人工智能学院

附录 · 储备主题

—进度提前 / Q&A 时启用；素材量 ≥ 1 小时

- ▶ 附录 A · 从 math 到 code 到 agent：能力阶梯与注入路径
- ▶ 附录 B · AI 的下半场：如何给模型一个能力——为什么先搭 benchmark
- ▶ 例：产品级 VLM (Vision-Language Model) / Seedream / 让大模型会做化学题

附录 A1 · 从 math 到 code 到 agent：三级能力阶梯

- ▶ **Math**: verifier = 答案/证明 kernel; RLVR 可规模化; 能力 = 搜索 + 自纠; bench: MATH/AIME (American Invitational Mathematics Examination) /IMO-style。
- ▶ **Code**: verifier = 编译器 + 单测; horizon 从函数拉长到 repo; bench: HumanEval → SWE-bench; 能力 = 长程规划 + 终端工具。
- ▶ **Agent**: verifier = 任务成功 (稀疏且慢); horizon = 小时到天; bench: tau-bench / terminal-bench / OSWorld; 能力 = 编排 + 上下文工程。
- ▶ 规律: **verifier 越弱、horizon 越长**, 「给能力」的重心越从 RL 奖励移向 harness 与 eval。

附录 A2 · 为什么是这个顺序: prior 复用与 eval 成本

- ▶ 每一级把上一级能力当 prior: math 推理 → code 推理 → agent 决策; 跳过阶梯的能力注入通常失败。
- ▶ eval 成本逐级上升 (秒 → 分钟 → 小时), 决定数据飞轮转速 ⇒ 决定哪些能力能被训练出来而不是被 prompt 出来。
- ▶ 所以下半场接到新能力时先问两件事: 它的 verifier 是什么? 它的 horizon 多长? —— 答案决定注入路径 (RLVR / SFT+ 工具 / harness+eval)。

附录 B1 · 如何给模型一个能力：为什么先搭 benchmark

- ▶ benchmark = 能力的**操作化定义**：同时给出 verifier、数据引擎入口与验收标准；没有它，训练是黑箱、进度不可测、recipe 无法工业化。
- ▶ 模板四步：定能力、搭 benchmark（含 anti-leak / anti-hack 设计） 数据引擎（合成/策展/蒸馏 + 难度课程） 训练排期（PT 续训/CT/SFT/RL，奖励设计） eval 驱动迭代（错误分析回流数据与 benchmark）。
- ▶ 呼应 Yao 的下半场判断：recipe 已工业化，**瓶颈在 evaluation 与 setup**——benchmark 的世界假设错，整个飞轮往错处转。

附录 B2 · 例：产品级 VLM (Vision-Language Model) 怎么训

- ▶ benchmark 先行：感知/OCR (Optical Character Recognition) /图表/多模态推理/产品真实 query 子集 + 幻觉负例；anti-hack：水印 OCR、图表轴扰动、近重复去漏。
- ▶ 数据引擎：大规模 caption/交错文档；**failure-driven** 难例挖掘；分辨率与长宽比课程。
- ▶ 训练排期：visual tokenizer+connector 续预训练 → 指令 SFT → 可验证子集 RLVR + 偏好对齐。
- ▶ 验收：产品指标（任务成功率/幻觉率）与 benchmark 对齐，而非学术分；bench 随产品迭代更新。

附录 B3 · 例：Seedream（图像生成）怎么训

- ▶ benchmark 先行：美学/文字渲染/人手人脸/identity (ID) 一致性/构图控制；人类偏好面板 + 自动代理指标双轨。
- ▶ 数据引擎：去重/去水印/美学打分/图文对齐打分的全量 curation；多粒度合成 caption。
- ▶ 训练排期：diffusion/flow-matching 大规模预训练 → 高分辨率续训 → 美学偏好对齐 → 文字渲染与版式专项数据。
- ▶ 验收：盲评 win-rate + 渲染准确率；held-out 偏好集防指标过拟合（metric hacking）。

附录 B4 · 例：让大模型会做化学题

- ▶ 为什么难：符号（方程/结构式）+ 空间（立体化学）+ 计算（热力学/动力学）三重负担；verifier 难搭。
- ▶ benchmark 先行：本科/竞赛化学 + 反应预测/逆合成子集；verifier：方程配平、RDKit 规范化、数值容差。
- ▶ 数据引擎：教材与题解、论文反应抽取、模拟/第一性计算合成 ground truth。
- ▶ 训练排期：领域续预训练 → 带解题模板 SFT → 可验证子集 RLVR；RDKit/物性计算作为工具补 agent 能力。
- ▶ 验收：分布外反应类型 + 误差传播测试（多步合成的累积错误）。